PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT

ATM TRANSACTION SEQUENCE SIMULATION

Submitted by: GANESHULA JAYA SAI VENKATESH

Roll Number: 26000899

Branch: CSE – Artificial Intelligence and Machine Learning

College: Dr. RVR & Dr. S. R. NRI Institute of Technology

Academic Year: 2026–2027

1. Problem Statement

An ATM (Automated Teller Machine) allows users to perform basic banking transactions such as depositing and withdrawing money. This project simulates a sequence of ATM transactions using Python. The program maintains the account balance, accepts multiple transactions, validates transaction amounts, rejects withdrawals when the available balance is insufficient, and records transaction details. Finally, it displays the transaction history and the initial and final account balances.

2. Objectives

The main objectives of the ATM Transaction Sequence Simulation are:

1. To simulate basic ATM transactions such as deposits and withdrawals.
2. To maintain and update the account balance after each successful transaction.
3. To validate transaction amounts and reject invalid amounts.
4. To prevent withdrawals when the available balance is insufficient.
5. To allow users to perform multiple transactions in a single session.
6. To store transaction details, including the transaction type, amount, and status.
7. To display the transaction history, initial balance, and final balance.

3. Assumptions

The following assumptions are considered in this ATM simulation:

1. The initial account balance is ₹1000.
2. The user can perform deposits and withdrawals in a single session.
3. The transaction amount must be greater than zero.
4. A withdrawal is successful only when the requested amount does not exceed the available balance.
5. Only the transaction types deposit and withdrawal are accepted.
6. The program continues accepting transactions until the user enters no.
7. All transaction details are stored in a list during the program execution.
8. The program simulates ATM transactions and does not connect to a real bank account.

4. Algorithm

1. Start the program.
2. Initialize the initial balance to ₹1000 and set the current balance equal to the initial balance.
3. Create an empty list to store transaction details.
4. Ask the user to enter the transaction type: deposit or withdrawal.
5. Ask the user to enter the transaction amount.
6. Check whether the amount is less than or equal to zero. If it is, reject the transaction as invalid.
7. If the transaction is a withdrawal, check whether the amount exceeds the available balance. If it does, reject the transaction; otherwise, deduct the amount from the balance.
8. If the transaction is a deposit, add the amount to the balance.
9. If the transaction type is invalid, reject the transaction.
10. Store the transaction type, amount, and status in the transaction history.
11. Ask the user whether they want to perform another transaction.
12. Repeat the transaction process if the user enters yes.
13. Display the initial balance, transaction history, and final balance.
14. Stop the program.

5. Pseudocode

```
START

SET initial_balance = 1000
SET balance = initial_balance
CREATE empty list transactions
SET choice = "yes"

WHILE choice = "yes"

  INPUT transaction_type
  INPUT amount

  IF amount <= 0 THEN
    DISPLAY "Invalid amount"
    SET status = "Rejected"

  ELSE IF transaction_type = "withdrawal" THEN

    IF amount > balance THEN
      DISPLAY "Insufficient balance"
      SET status = "Rejected"
    ELSE
      balance = balance - amount
      DISPLAY "Withdrawal successful"
      SET status = "Successful"
    END IF
```

```
    ELSE IF transaction_type = "deposit" THEN
        balance = balance + amount
        DISPLAY "Deposit successful"
        SET status = "Successful"

    ELSE
        DISPLAY "Invalid transaction type"
        SET status = "Rejected"
    END IF

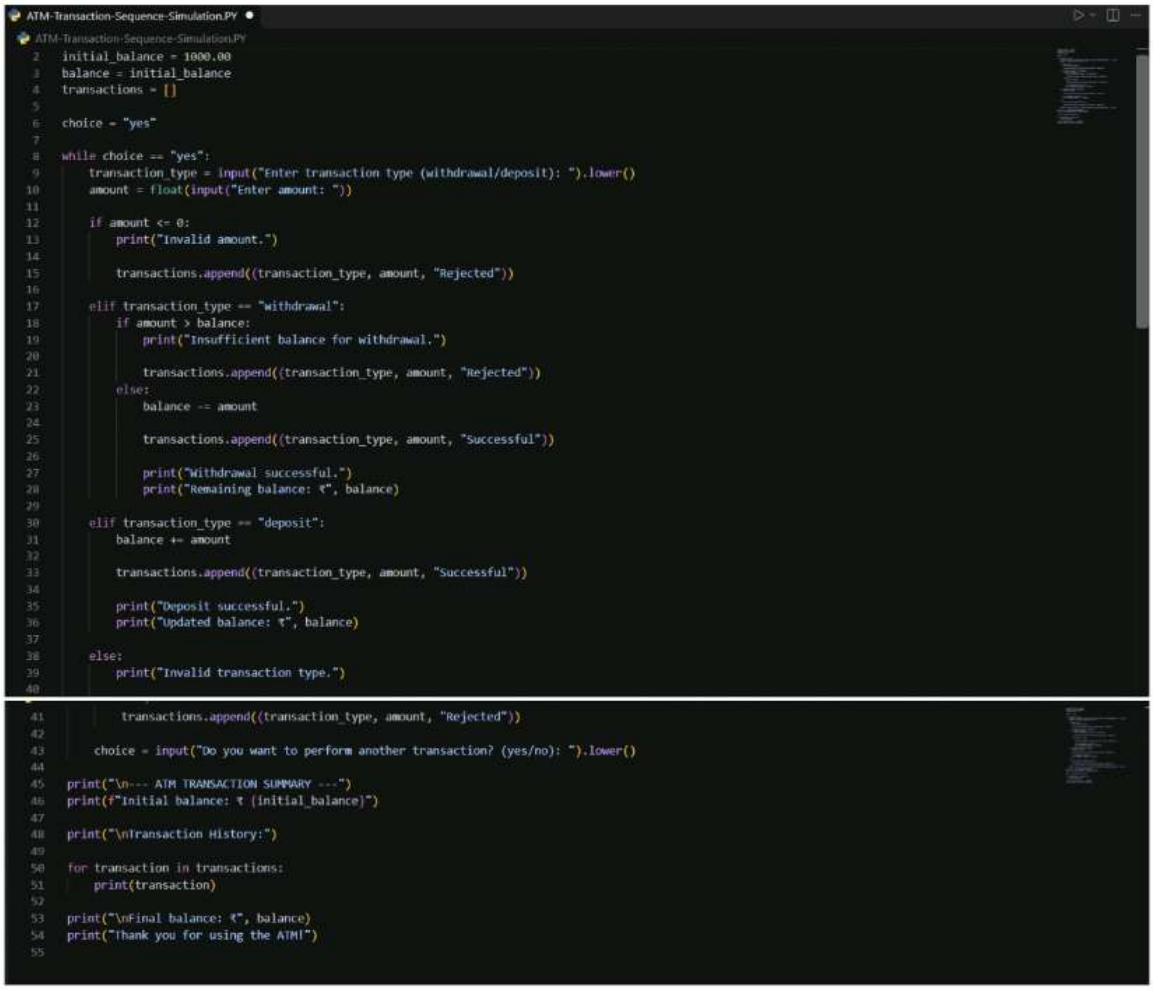
    ADD (transaction_type, amount, status)
        TO transactions

    INPUT choice
END WHILE

DISPLAY initial_balance
DISPLAY all transactions
DISPLAY balance

STOP
```

6. Python Source Code



```
ATM-Transaction-Sequence-Simulation.PY
initial_balance = 1000.00
balance = initial_balance
transactions = []

choice = "yes"

while choice == "yes":
    transaction_type = input("Enter transaction type (withdrawal/deposit): ").lower()
    amount = float(input("Enter amount: "))

    if amount <= 0:
        print("Invalid amount.")

        transactions.append((transaction_type, amount, "Rejected"))

    elif transaction_type == "withdrawal":
        if amount > balance:
            print("Insufficient balance for withdrawal.")

            transactions.append((transaction_type, amount, "Rejected"))
        else:
            balance -= amount

            transactions.append((transaction_type, amount, "Successful"))

            print("Withdrawal successful.")
            print("Remaining balance: ₹", balance)

    elif transaction_type == "deposit":
        balance += amount

        transactions.append((transaction_type, amount, "Successful"))

        print("Deposit successful.")
        print("Updated balance: ₹", balance)

    else:
        print("Invalid transaction type.")

        transactions.append((transaction_type, amount, "Rejected"))

    choice = input("Do you want to perform another transaction? (yes/no): ").lower()

print("\n--- ATM TRANSACTION SUMMARY ---")
print(f"Initial balance: ₹ {initial_balance}")

print("\nTransaction History:")

for transaction in transactions:
    print(transaction)

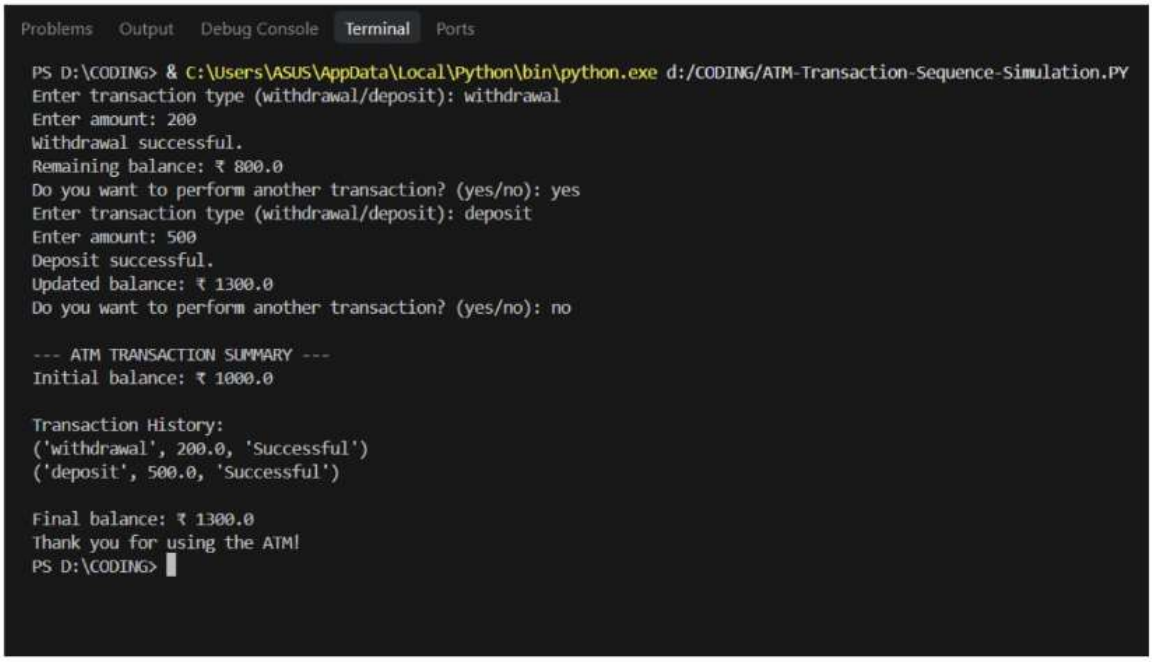
print("\nFinal balance: ₹", balance)
print("Thank you for using the ATM!")
```

7. Test Cases

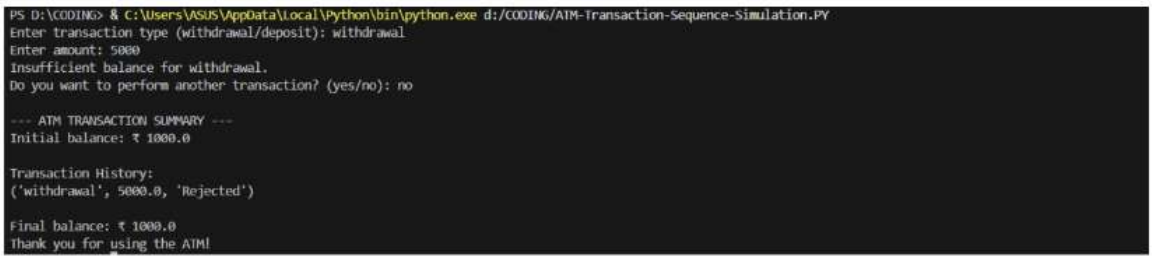
s.no	Transaction	Input amount	Expected result
1	Withdrawal	200	Successful; balance becomes ₹800
2	Deposit	500	Successful; balance becomes ₹1,300
3	Withdrawal	5000	Rejected; insufficient balance
4	Deposit	0	Rejected; invalid amount
5	Invalid type	100	Rejected; invalid transaction type

8. Output Screenshots

Screenshot 1: Successful Deposit and Withdrawal Transactions



Screenshot 2: Rejected Transactions



9. Conclusion

The ATM Transaction Sequence Simulation was successfully developed using Python. The program performs deposits and withdrawals, maintains the account balance, validates transaction amounts, and rejects invalid transactions and withdrawals when sufficient funds are not available. It also supports multiple transactions in a single session and stores transaction details in a list. Finally, the program displays the transaction history along with the initial and final balances. This project helped in understanding Python concepts such as variables, conditional statements, loops, lists, tuples, and user input.